

CAPITOLUL 1

CAPITOLUL 1 – Cursul 2

Shell Scripting – Partea I

Scripts

- Scripturile conțin o serie de comenzi.
 - Un interpret execută comenzile din script.
 - Orice comandă care poate fi tastată în linia de comandă poate fi pusă într-un script.
 - Scripturile sunt foarte utile pentru automatizarea task-urilor.
-

Exemplu de script – script.sh

```
#!/bin/bash  
echo "Scripting is fun!"
```

Rulare:

```
chmod 755 script.sh  
./script.sh
```

Output:

Scripting is fun!

Shebang (#!)

- Shebang-ul este prima linie dintr-un script.
- Specifică interpretul care va executa scriptul.

Exemple:

csh:

```
#!/bin/csh
echo "This script uses csh as the interpreter."
```

ksh:

```
#!/bin/ksh
echo "This script uses ksh as the interpreter."
```

zsh:

```
#!/bin/zsh
echo "This script uses zsh as the interpreter."
```

Script rulat în background – sleepy.sh

```
#!/bin/bash
sleep 90
```

Rulare în background:

```
./sleepy.sh &
```

Shell-ul afișează un PID (Process ID).

Verificare proces:

```
ps -fp <PID>
```

Interpretul execută scriptul

Exemplu:

```
/tmp/sleepy.sh &
```

Verificare:

```
ps -fp <PID>
```

Se observă că scriptul este rulat de:

/bin/bash /tmp/sleepy.sh

Shebang sau nu?

- Dacă un script nu conține shebang, comenzile sunt executate folosind shell-ul curent.
- Este posibil să funcționeze sau nu.
- Shell-uri diferite au sintaxă ușor diferită.

Recomandare: folosește întotdeauna shebang.

Mai mult decât shell scripts

Scripturile nu sunt doar Bash.

Exemplu Python:

```
#!/usr/bin/python  
print "This is a Python script."
```

Rulare:

```
chmod 755 hi.py  
./hi.py
```

Output:

This is a Python script.

Variabile

- Variabilele sunt locații de stocare cu nume.
- Sunt perechi nume–valoare.
- Sintaxă:

VARIABLE_NAME="Value"

- Variabilele sunt case-sensitive.
 - Prin convenție, variabilele sunt scrise cu litere mari.
-

Utilizarea variabilelor

```
#!/bin/bash
MY_SHELL="bash"
echo "I like the $MY_SHELL shell."
```

Formă recomandată:

```
echo "I like the ${MY_SHELL} shell."
```

Exemplu de utilizare greșită a variabilelor

```
#!/bin/bash
MY_SHELL="bash"
echo "I am $MY_SHELLing on my keyboard."
```

Output:

I am on my keyboard.

Explicație: shell-ul caută variabila `MY_SHELLing`, care nu există.

Atribuirea rezultatului unei comenzi unei variabile

```
#!/bin/bash
SERVER_NAME=$(hostname)

echo "You are running this script on ${SERVER_NAME}."
```

Output:

You are running this script on linuxsvr

Shell Scripting – Partea II

if / else

```
if [ condition-is-true ]
then
    command N
else
    command N
fi
```

if / elif / else

```
if [ condition-is-true ]
then
    command N
elif [ condition-is-true ]
then
    command N
else
    command N
fi
```

Exemplu if / elif / else

```
#!/bin/bash

MY_SHELL="csh"

if [ "$MY_SHELL" = "bash" ]
then
    echo "You seem to like the bash shell."
elif [ "$MY_SHELL" = "csh" ]
then
    echo "You seem to like the csh shell."
else
    echo "You don't seem to like the bash or csh shells."
fi
```

Variable Names

Valid:

```
FIRST3LETTERS="ABC"  
FIRST_THREE_LETTERS="ABC"  
firstThreeLetters="ABC"
```

Invalid:

```
3LETTERS="ABC"  
first-three-letters="ABC"  
first@three@letters="ABC"
```

Tests

Syntax:

```
[ condition-to-test-for ]
```

Example:

```
[ -e /etc/passwd ]
```

File operators (tests)

- **-d FILE** – True dacă fișierul este director
 - **-e FILE** – True dacă fișierul există
 - **-f FILE** – True dacă fișierul există și este fișier normal
 - **-r FILE** – True dacă fișierul poate fi citit de tine
 - **-s FILE** – True dacă fișierul există și nu este gol
 - **-w FILE** – True dacă fișierul poate fi scris de tine
 - **-x FILE** – True dacă fișierul este executabil de tine
-

String operators (tests)

- `-z STRING` – True dacă string-ul este gol
 - `-n STRING` – True dacă string-ul nu este gol
 - `STRING1 = STRING2`
→ True dacă string-urile sunt egale
 - `STRING1 != STRING2`
→ True dacă string-urile nu sunt egale
-

Arithmetic operators (tests)

- `arg1 -eq arg2` – True dacă arg1 este egal cu arg2
 - `arg1 -ne arg2` – True dacă arg1 nu este egal cu arg2
 - `arg1 -lt arg2` – True dacă arg1 este mai mic decât arg2
 - `arg1 -le arg2` – True dacă arg1 este mai mic sau egal cu arg2
 - `arg1 -gt arg2` – True dacă arg1 este mai mare decât arg2
 - `arg1 -ge arg2` – True dacă arg1 este mai mare sau egal cu arg2
-

Making Decisions – The if statement

```
if [ condition-is-true ]
then
    command 1
    command 2
    command N
fi
```

Exemplu if simplu

```
#!/bin/bash
```

```
MY_SHELL="bash"
```

```
if [ "$MY_SHELL" = "bash" ]  
then  
    echo "You seem to like the bash shell."  
fi
```

Output:

You seem to like the bash shell.

CAPITOLUL 1 – Cursul 4

Shell Scripting – Partea II (for loop, parametri)

For loop – sintaxă generală

```
for VARIABLE_NAME in ITEM_1 ITEM_N  
do  
    command 1  
    command 2  
    command N  
done
```

Exemplu simplu – for loop

```
#!/bin/bash
```

```
COLORS="red green blue"
```

```
for COLOR in $COLORS
do
    echo "COLOR: $COLOR"
done
```

Exemplu practic – redenumire fișiere

Conținut director:

```
$ ls
bear.jpg man.jpg pig.jpg rename-pics.sh
```

Rulare script:

```
$ ./rename-pics.sh
```

Output:

```
Renaming bear.jpg to 2015-03-06-bear.jpg
Renaming man.jpg to 2015-03-06-man.jpg
Renaming pig.jpg to 2015-03-06-pig.jpg
```

După rulare:

```
$ ls
2015-03-06-bear.jpg
2015-03-06-man.jpg
2015-03-06-pig.jpg
rename-pics.sh
```

Positional Parameters

Rulare script:

```
$ script.sh parameter1 parameter2 parameter3
```

Parametri:

```
$0 : script.sh
$1 : parameter1
$2 : parameter2
$3 : parameter3
```

Exemplu – folosirea parametrilor poziționali

```
#!/bin/bash

echo "Executing script: $0"
echo "Archiving user: $1"

# Lock the account
passwd -l $1

# Create an archive of the home directory.
tar cf /archives/${1}.tar.gz /home/${1}
```

For loop cu \$@ (toți parametrii)

```
#!/bin/bash

echo "Executing script: $0"

for USER in $@
do
    echo "Archiving user: $USER"

    # Lock the account
    passwd -l $USER

    # Create an archive of the home directory.
    tar cf /archives/${USER}.tar.gz /home/${USER}
done
```

Rulare script cu mai mulți utilizatori

```
$ ./archive_user.sh chet joe
```

Output:

```
Executing script: ./archive_user.sh
Archiving user: chet
passwd: password expiry information changed.
tar: Removing leading '/' from member names
Archiving user: joe
passwd: password expiry information changed.
tar: Removing leading '/' from member names
```

Rulare script fără parametri (input de la tastatură)

```
$ ./archive_user.sh
```

Output:

```
Enter a user name: mitch  
Archiving user: mitch  
passwd: password expiry information changed.  
tar: Removing leading '/' from member names
```

Summary

```
#!/path/to/interpreter
```

```
VARIABLE_NAME="Value"
```

```
$VARIABLE_NAME  
${VARIABLE_NAME}
```

```
VARIABLE_NAME=$(command)
```

```
#####  
# EXIT STATUS / RETURN CODE  
#####
```

```
# Orice comandă returnează un exit status  
# Interval: 0 – 255  
# 0 = succes  
# ≠0 = eroare  
# Folosit pentru verificarea erorilor
```

```
#####  
# Verificarea Exit Status  
#####
```

```
# $? conține codul de retur al ultimei comenzi
```

```
ls /not/here  
echo "$?"
```

```
#####  
# Exemplu cu ping  
#####
```

```
HOST="google.com"  
ping -c 1 $HOST
```

```
if [ "$?" -eq "0" ]  
then  
echo "$HOST reachable."  
else  
echo "$HOST unreachable."  
fi
```

```
#####  
# Salvarea codului de retur  
#####
```

```
HOST="google.com"  
ping -c 1 $HOST  
RETURN_CODE=$?
```

```
if [ "$RETURN_CODE" -ne "0" ]
then
echo "$HOST unreachable."
fi
```

```
#####
# && și ||
#####
```

```
# && = AND (rulează a doua comandă doar dacă prima reușește)
mkdir /tmp/bak && cp test.txt /tmp/bak/
```

```
# || = OR (rulează a doua comandă dacă prima eșuează)
cp test.txt /tmp/bak/ || cp test.txt /tmp/
```

```
# Exemplu
#!/bin/bash
```

```
HOST="google.com"
ping -c 1 $HOST || echo "$HOST unreachable."
```

```
#####
# SEMICOLON (;)
#####
```

```
# Separă comenzi care vor fi executate indiferent de rezultat
cp test.txt /tmp/bak/ ; cp test.txt /tmp
```

```
#####
# Exemplu complet cu exit
#####
```

```
#!/bin/bash
```

```
HOST="google.com"
ping -c 1 $HOST
```

```
if [ "$?" -ne "0" ]
then
```

```
echo "$HOST unreachable."  
exit 1  
fi
```

```
exit 0
```

CURS 4:

Why use functions? (Keep it DRY!)

- Don't repeat yourself! Don't repeat yourself!
 - Write once, use many times.
 - Reduces script length.
 - Single place to edit and troubleshoot.
 - Easier to maintain.
-

Functions

- If you're repeating yourself, use a function.
 - Reusable code.
 - Must be defined before use.
 - Has parameter support.
-

Creating a function

```
function function_name() {  
    # Code goes here.  
}
```

Sau forma echivalentă:

```
function_name() {  
    # Code goes here.  
}
```

Calling a function

```
#!/bin/bash
```

```
function hello() {  
    echo "Hello!"  
}
```

```
hello
```

Functions can call other functions

```
#!/bin/bash
```

```
function hello() {  
    echo "Hello!"  
    now  
}
```

```
function now() {  
    echo "It's $(date +%r)"  
}
```

```
hello
```

Do NOT do this

 Apelarea unei funcții înainte ca ea să fie definită:

```
#!/bin/bash
```

```
function hello() {  
    echo "Hello!"  
    now  
}
```

```
hello
```

```
function now() {  
    echo "It's $(date +%r)"  
}
```

👉 Funcțiile **trebuie definite înainte de a fi apelate**.

Positional Parameters (Functions)

- Functions can accept parameters.
 - First parameter → `$1`
 - Second parameter → `$2`, etc.
 - `$@` conține toți parametrii.
 - La fel ca în shell scripts.
 - `$0` este **scriptul**, nu numele funcției.
-

Exemplu – parametri într-o funcție

```
#!/bin/bash
```

```
function hello() {  
    echo "Hello $1"  
}
```

```
hello Jason
```

Output:

```
Hello Jason
```

Variable Scope

- By default, variables are **global**.
- Variables must be defined before use.

Exemplu – variabilă globală

```
GLOBAL_VAR=1
```

```
# GLOBAL_VAR este disponibilă în funcție  
my_function
```

Variabilă globală definită în funcție

```
#!/bin/bash
```

```
my_function() {  
    GLOBAL_VAR=1  
}
```

```
# GLOBAL_VAR nu este disponibilă încă  
echo $GLOBAL_VAR
```

```
my_function
```

```
# GLOBAL_VAR este acum disponibilă  
echo $GLOBAL_VAR
```

Local Variables

- Pot fi accesate **doar în interiorul funcției**.
 - Se creează cu keyword-ul `local`.
 - Doar funcțiile pot avea variabile locale.
 - Best practice: păstrează variabilele **locale** în funcții.
-

Exemplu – variabilă locală

```
#!/bin/bash
```

```
my_function() {  
    local LOCAL_VAR=1  
    echo $LOCAL_VAR  
}
```

```
my_function
```

```
# Aceasta NU va functiona:  
echo $LOCAL_VAR
```

Summary – Functions

```
# Definire funcție  
function_name() {  
    commands  
}
```

```
# Apel funcție  
function_name
```

```
# Parametri  
$1 $2 $@
```

```
# Variabile locale  
local VAR=value
```

Exit Status (Return Codes)

Exit Status – general

- Functions have an exit status.
 - Exit status poate fi:
 - **Explicit**
 - `return <RETURN_CODE>`
 - **Implicit**
 - Exit status al **ultimei comenzi executate** în funcție.
-

Exit Status – reguli

- Codurile valide sunt între **0 și 255**
 - **0 = success**
 - Orice valoare $\neq 0$ = **failure**
 - **\$?** = exit status al **ultimei comenzi** sau funcții rulate
-

Verificarea exit status

```
my_function  
echo $?
```

Exemplu – funcție fără return explicit

```
backup_file() {  
  if [ -f $1 ]  
  then  
    BACK="/tmp/${basename ${1}}.$(date +%F).$$"  
    echo "Backing up $1 to ${BACK}"  
    cp $1 $BACK  
  fi
```

```
}
```

👉 Exit status-ul funcției va fi exit status-ul comenzii `cp`.

Exemplu complet – funcție cu return explicit

```
#!/bin/bash
```

```
function backup_file() {  
    if [ -f $1 ]  
    then  
        local BACK="/tmp/${basename ${1}}.$(date +%F).$$"  
        echo "Backing up $1 to ${BACK}"  
        cp $1 $BACK  
    else  
        # Fișierul nu există  
        return 1  
    fi  
}
```

Folosirea exit status-ului funcției

```
backup_file /etc/hosts
```

```
# Luăm decizia pe baza exit status-ului  
if [ $? -eq 0 ]  
then  
    echo "Backup succeeded!"  
else  
    echo "Backup failed!"  
    # Oprim scriptul cu exit non-zero  
    exit 1  
fi
```

Rezumat – Exit Status

```
# Return explicit din funcție  
return 0  
return 1
```

Exit status implicit
= statusul ultimei comenzi

Verificare
\$?

În afara funcțiilor (script)
exit 0
exit 1

CURS 5:

Shell Script Order and Checklist

1. Shebang

```
#!/bin/bash
```

2. Comments / file header

```
# Script name: example.sh  
# Description: Example shell script  
# Author: ...  
# Date: ...
```

3. Global variables

```
GLOBAL_VAR=value
```

4. Functions

- Use **local variables** inside functions

```
5. my_function() {  
    local LOCAL_VAR=value  
    echo "$LOCAL_VAR"
```

```
}
```

6. Main script contents

```
my_function  
echo "Main script logic here"
```

7. Exit with an exit status

- Use `exit <STATUS>` at various exit points

8. exit 0

Checklist – rapid (de examen)

- Are shebang?
- Are comentarii / header?
- Variabilele globale sunt sus?
- Funcțiile sunt definite înainte de utilizare?
- Variabilele din funcții sunt `local`?
- Logica principală este la final?
- Scriptul se termină cu `exit`?

Wildcards (Globbing)

Ce sunt wildcards?

- Un **caracter** sau **șir de caractere** folosit pentru **pattern matching**.
- **Globbing** = shell-ul extinde wildcard-ul într-o listă de **fișiere și/sau directoare (paths)**.
- Wildcards pot fi folosite cu majoritatea comenzilor:

- `ls`
- `rm`
- `cp`
- `mv`
- etc.

* – zero sau mai multe caractere

- * → orice
- *.txt → toate fișierele .txt
- a* → fișiere care încep cu a
- a*.txt → fișiere .txt care încep cu a

? – exact un caracter

- ? .txt → un caracter + .txt
 - a? → a + un caracter
 - a? .txt → a + un caracter + .txt
-

Character Classes – []

[] – exact un caracter din set

- Potrivește **un singur caracter** dintre cele din paranteze.

[aeiou] → o vocală

ca[nt]* → can*, cat*

Exemple potrivite:

can

cat

candy

catch

Character Classes – negare [!]

[!] – exact un caracter care **NU** este în set

[!aeiou]*

Exemple potrivite:

baseball

cricket

Ranges – intervale

Sintaxă

- Două caractere separate de - într-o clasă.

[a-g]* → începe cu a, b, c, d, e, f sau g

[3-6]* → începe cu 3, 4, 5 sau 6

Named Character Classes

Clase predefinite (POSIX):

[[:alpha:]] → litere
[[:alnum:]] → litere + cifre
[[:digit:]] → cifre
[[:lower:]] → litere mici
[[:upper:]] → litere mari
[[:space:]] → spații

Exemplu:

[[[:digit:]]*
[[[:alpha:]]*

Escaping wildcard characters

\ – escape character

- Folosește \ dacă vrei să **potrivești wildcard-ul ca simbol literal**, nu ca pattern.

*\? → fișiere care se termină cu ?

Exemplu fișier:

done?

Rezumat rapid (de examen)

* → zero sau mai multe caractere
? → exact un caracter
[] → un caracter din set
[!] → un caracter care NU este în set
[a-z] → range
[:digit:] etc. → clase predefinite
\ → escape wildcard

De ce folosim wildcards (*)

Wildcard-urile sunt folosite ca să lucrezi **dintr-o singură comandă** cu mai multe fișiere care respectă un tipar.

👉 ***.html** = toate fișierele care se termină în **.html**

De exemplu:

- **about.html**
- **contact.html**
- **index.html**

Shell-ul (bash) înlocuiește wildcard-ul cu lista reală de fișiere înainte să ruleze bucla **for**.

Ce face scriptul tău, pas cu pas

```
cd /var/www
```

➡ Te muți în directorul **/var/www**

```
for FILE in *.html
```

➡ Bash extinde ***.html** la:

```
about.html contact.html index.html  
echo "Copying $FILE"
```

➡ Afișează numele fișierului curent

```
cp $FILE /var/www-just-html
```

➡ Copiază fișierul în alt director

De ce apar două tipuri de output

1 Fără cale completă

```
Copying about.html  
Copying contact.html
```

Copying index.html

✓ Pentru că ești deja în `/var/www`, iar `$FILE` conține **doar numele fișierului**

2 Cu cale completă

Copying /var/www/about.html

Copying /var/www/contact.html

Copying /var/www/index.html

✓ Asta se întâmplă dacă:

- nu ai făcut `cd /var/www`, sau
- wildcard-ul a fost folosit cu o cale:

for FILE in /var/www/*.html

În cazul ăsta, `$FILE` conține **path-ul complet**, nu doar numele.

CURS7:

Ce este **case**

case este **alternativa mai curată la multe **if** / **elif**** atunci când verifici **o singură variabilă** cu mai multe valori posibile.

În loc de:

```
if [ "$VAR" = "one" ]; then ...  
elif [ "$VAR" = "two" ]; then ...  
elif [ "$VAR" = "three" ]; then ...  
fi
```

folosești:

```
case "$VAR" in  
  one) ... ;;  
  two) ... ;;  
  three) ... ;;  
esac
```

👉 mai lizibil, mai scurt, mai clar.

Structura generală

```
case "VALOARE" in  
  pattern1)  
    comenzi  
    ;;  
  pattern2)  
    comenzi  
    ;;  
  *)  
    fallback (default)  
    ;;  
esac
```

🔑 lucruri importante:

- `;;` = oprește execuția aceluși caz
 - `*` = default (ca `else`)
 - `esac` = `case` scris invers (așa se închid blocurile în bash)
-

Exemplu 1 – script start / stop (din slide)

```
case "$1" in
start)
    /usr/sbin/sshd
    ;;
stop)
    kill $(cat /var/run/sshd.pid)
    ;;
*)
    echo "Usage: $0 start|stop"
    exit 1
    ;;
esac
```

- 👉 `$1` = primul argument din linia de comandă
 - 👉 `./script.sh start` → intră pe `start`)
 - 👉 orice altceva → intră pe `*`)
-

Exemplu 2 – input de la utilizator (Y / N)

```
read -p "Enter y or n: " ANSWER
```

```
case "$ANSWER" in
[yY][yY][eE][sS])
    echo "You answered yes."
    ;;
[nN][nN][oO])
    echo "You answered no."
    ;;
*)
    echo "Invalid answer."
    ;;
esac
```

esac

Ce e nou aici?

- `[yY]` → y sau Y
- `[yY][eE][sS]` → yes / YES / yEs etc.
- `|` = OR între pattern-uri

Variantă mai simplă (wildcard)

```
case "$ANSWER" in
[yY]*)
    echo "You answered yes."
    ;;
*)
    echo "You answered something else."
    ;;
esac
```

👉 `[yY]*` = începe cu y sau Y, restul nu contează

Ce este logging-ul

Logging = jurnal de evenimente

Răspunde la: **cine, ce, când, unde, de ce**

De ce e important:

- output-ul din terminal **dispare**
 - scripturile rulează **fără utilizator** (cron)
 - ai nevoie de **istoric** pentru debug / audit
-

Syslog – baza sistemului de loguri

Linux folosește **syslog** ca standard de logging.

Syslog clasifică mesajele prin:

❶ Facility (cine a generat mesajul)

Exemple:

- **kern** – kernel
- **user** – aplicații user
- **daemon** – servicii
- **auth** – autentificare
- **mail**
- **local0 ... local7** – pentru scripturi custom 🙌 (foarte important)

❷ Severity (cât de grav)

Ordine de la grav → minor:

emerg
alert
crit
err
warning
notice
info
debug

Unde ajung logurile

Depinde de configurație (`rsyslog`, `syslog-ng`), dar uzual:

- `/var/log/messages`
 - `/var/log/syslog`
 - `/var/log/auth.log`
-

logger – unealta pentru scripturi

`logger` trimite mesaje **direct** în `syslog`.

Simplu

`logger "Message"`

➡ default: `user.notice`

Specifici facility + severity

`logger -p local0.info "Message"`

➡ `local0` = script

➡ `info` = nivel

Tag (numele scriptului)

```
logger -t myscript -p local0.info "Message"
```

Output:

```
myscript: Message
```

Afișare și pe ecran

```
logger -s -p local0.info "Message"
```

➡ apare și în terminal, și în log

Include PID (foarte util)

```
logger -i -t myscript "Message"
```

Output:

```
myscript[12986]: Message
```

Pattern standard de logging în scripturi

```
logger -t "$0" -p local0.info "Started script"
```

✓ `$0` = numele scriptului

✓ `local0` = rezervat pentru aplicația ta

Funcție custom de logging (din slide)

Ideea ei:

- definești nivel (`ERROR`, `INFO`)
- adaugi **timestamp**

- afișezi doar ce contează (ex: erori)

Conceptual:

```
logit ERROR "Something failed"
```

```
logit INFO "Process started"
```

👉 foarte util în scripturi mari

CURSUL 9 – WHILE LOOPS ÎN BASH

Formatul general al unei bucle while

Structura de bază:

```
while [ CONDITION_IS_TRUE ]  
do  
    command1  
    command2  
    commandN  
done
```

 Explicație:

- `while` – începe bucla
- `[ condiție ]` – expresie test
- `do` – începe blocul de cod
- `done` – închide bucla

 Bucla rulează cât timp condiția este TRUE.

2 Infinite Loops (Bucle infinite)

O buclă infinită este una care nu se oprește niciodată:

```
while true
do
    command
    sleep 1
done
```

Explicație:

- `true` este mereu adevărat
 - `sleep 1` previne consumul excesiv de CPU
 - Bucla rulează la infinit până la:
 - CTRL + C
 - sau un `break`
-

3 Exemplu – Rulează de 5 ori

```
INDEX=1
```

```
while [ $INDEX -lt 6 ]
do
    echo "Creating project-${INDEX}"
    mkdir /usr/local/project-${INDEX}
    ((INDEX++))
done
```



Ce face:

1. Inițializează `INDEX=1`
2. Verifică dacă `INDEX < 6`

3. Creează directoare:

- project-1
- project-2
- project-3
- project-4
- project-5

4. Se oprește când INDEX devine 6

4 Verificare input utilizator

Exemplu:

```
while [ "$CORRECT" != "y" ]
do
  read -p "Enter your name: " NAME
  read -p "Is ${NAME} correct? " CORRECT
done
```

 Ce face:

- Cere numele utilizatorului
- Întreabă dacă este corect
- Dacă utilizatorul nu introduce "y", repetă bucla

Foarte util pentru:

- validare input
- confirmări
- formulare interactive

5 Citirea unui fișier linie cu linie

Exemplu de bază:

```
LINE_NUM=1
```

```
while read LINE
do
    echo "${LINE_NUM}: ${LINE}"
    ((LINE_NUM++))
done < /etc/fstab
```

 Ce face:

- Citește fiecare linie din `/etc/fstab`
- Afișează:
 - numărul liniei
 - conținutul liniei

6 Folosirea grep + while

```
grep xfs /etc/fstab | while read LINE
do
    echo "xfs: ${LINE}"
done
```

 Ce face:

- Caută liniile care conțin "xfs"
 - Le procesează una câte una
-

7 Citirea mai multor coloane

```
FS_NUM=1
```

```
grep xfs /etc/fstab | while read FS MP REST
do
    echo "${FS_NUM}: file system: ${FS}"
    echo "${FS_NUM}: mount point: ${MP}"
    ((FS_NUM++))
done
```

 Explicație:

`read FS MP REST` înseamnă:

- FS → primul câmp
- MP → al doilea câmp
- REST → restul liniei

Foarte util pentru:

- procesare fișiere de configurare
- extragere coloane

CURSUL 10 – DEBUGGING ÎN BASH + DOS vs LINUX FILE TYPES

1 Why Debug?

Ce este un bug?

Un **bug** este o eroare în script.

De ce facem debugging?

- Analizăm ce se întâmplă în interiorul scriptului
- Identificăm cauza comportamentului neașteptat
- Corectăm erorile

Debugging = înțelegerea exactă a execuției scriptului.

2 Built-in Debugging Help

◆ Opțiunea **-x**

`#!/bin/bash -x`

sau

`set -x`

Ce face **-x**?

- Afișează fiecare comandă înainte să fie executată
- După substituții și expansiuni
- Se mai numește:
 - x-trace
 - tracing
 - print debugging

Oprirea debugging-ului:

set +x

◆ Exemplu -x

Script:

```
#!/bin/bash -x
```

```
TEST_VAR="test"  
echo "$TEST_VAR"
```

Output:

```
+ TEST_VAR=test  
+ echo test  
test
```

Observație:

- Liniile cu + sunt comenzi executate
 - Ultima linie este output-ul real
-

3 Opțiunea -e

◆ Exit on error

```
#!/bin/bash -e
```

Ce face?

- Scriptul se oprește imediat dacă apare o eroare

Exemplu:

```
#!/bin/bash -e
```

```
FILE_NAME="/not/here"  
ls $FILE_NAME  
echo $FILE_NAME
```

Output:

ls: cannot access /not/here: No such file or directory

Scriptul se oprește.

Linia `echo $FILE_NAME` NU mai rulează.

◆ Combinarea opțiunilor

Toate sunt echivalente:

```
#!/bin/bash -ex  
#!/bin/bash -xe  
#!/bin/bash -e -x  
#!/bin/bash -x -e
```

4 Opțiunea -v

```
#!/bin/bash -v
```

Ce face?

- Afișează liniile scriptului exact cum sunt citite
 - Nu după expansiune (diferență față de -x)
-

5 Manual Debugging

Poți crea propriul sistem de debugging.

◆ Variabilă DEBUG

```
DEBUG=true
if $DEBUG
then
    echo "Debug mode ON."
else
    echo "Debug mode OFF."
fi
```

◆ Funcție debug

```
debug() {
    echo "Executing: $@"
    $@
}
```

Utilizare:

```
debug ls
debug pwd
```

Foarte util pentru:

- logare controlată
 - debugging selectiv
-

6 Manual Copy & Paste Debugging

- Deschizi un al doilea terminal
- Copiezi comenzile una câte una
- Poți activa `set -x` în shell

Metodă simplă dar eficientă.

7 Syntax Highlighting

Erorile de sintaxă sunt foarte comune:

- typo
- paranteze lipsă
- ghilimele lipsă

Folosește editor cu syntax highlighting:

- vi / vim
- emacs
- nano
- gedit
- kate
- geany

Ajută enorm la detectarea erorilor vizuale.

8 PS4 – Control pentru -x

PS4 controlează ce apare înaintea fiecărei linii când folosești -x.

Valoarea implicită:

+

Poți personaliza:

```
PS4='+ ${BASH_SOURCE}:${LINENO} '
```

Astfel vezi:

- numele fișierului
- numărul liniei

Extrem de util pentru scripturi mari.

9 DOS vs Linux (Unix) File Types

♦ CRLF vs LF

Windows:

- CRLF (Carriage Return + Line Feed)

Linux:

- LF (Line Feed)
-

♦ Detectarea caracterelor Windows

```
cat -v script.sh
```

Dacă vezi:

```
#!/bin/bash^M
```

^M = CR (carriage return)

Scriptul poate da eroare:

```
bad interpreter: /bin/bash^M
```

◆ Verificare tip fișier

file script.sh

Exemplu problematic:

script.sh: Bourne-Again shell script, ASCII text executable, with CRLF line terminators

◆ Conversie DOS → UNIX

dos2unix script.sh

Verificare după conversie:

script.sh: Bourne-Again shell script, ASCII text executable

10 Cum apare problema CRLF?

- Scrii scriptul în Windows
- Îl urci pe Linux
- Copiezi din browser în terminal
- Editorul folosește CRLF

CAPITOLUL 2

Capitolul 2 – Cursul 2

Virtualizare cu VirtualBox + Vagrant

1 Virtualization Software

Ce este virtualizarea?

Virtualizarea îți permite să rulezi un sistem de operare (Linux) în interiorul altui sistem (Windows/Mac/Linux).

Adică:

- Ai Windows instalat
 - Rulezi Linux într-o mașină virtuală (VM)
-

◆ VirtualBox

- Website: <https://virtualbox.org>
- Gratuit
- Ușor de instalat
- Funcționează pe:
 - Windows
 - Mac
 - Linux
- Perfect pentru laboratorul nostru

VirtualBox este motorul care creează mașina virtuală.

2 VirtualBox Guest Additions

Guest Additions îmbunătățesc experiența VM-ului.

Permit:

- Shared folders (folder comun între host și VM)
- Shared clipboard (copy/paste între host și VM)
- Performanță video mai bună
- Integrare mouse mai bună

Recomandat să le instalezi.

3 Vagrant

Ce este Vagrant?

Vagrant automatizează crearea și configurarea mașinilor virtuale.

În loc să configurezi manual VM-ul în VirtualBox, Vagrant face totul din terminal.

4 Vagrant Boxes

Ce este o "box"?

Box = imagine de sistem de operare preconfigurată.

Exemplu:

```
vagrant box add jasonc/centos7
```

Asta descarcă o imagine CentOS 7.

5 Vagrant Project

Un proiect Vagrant = folder care conține un fișier numit:

◆ Crearea proiectului

```
mkdir vm1  
cd vm1
```

◆ Inițializare

```
vagrant init jasonc/centos7
```

Se creează automat fișierul **Vagrantfile**.

6 Vagrant Up

```
vagrant up
```

Ce face:

- Importă box-ul în VirtualBox
 - Creează mașina virtuală
 - O pornește
 - Rulează în headless mode (fără interfață grafică)
-

7 Conectare prin SSH

```
vagrant ssh
```

Intră în mașina virtuală prin SSH.

Pentru a ieși:

```
exit
```

⚠ Pe Windows ai nevoie de SSH client
(Se instalează automat cu Git Bash)

8 Comenzi importante Vagrant

Comandă	Ce face
<code>vagrant up</code>	Pornește VM
<code>vagrant halt</code>	Oprește VM
<code>vagrant suspend</code>	Pune VM în sleep
<code>vagrant resume</code>	Repornește VM din suspend
<code>vagrant destroy</code>	Șterge VM
<code>vagrant</code>	Listează opțiuni

9 Vagrantfile

Exemplu de configurare:

```
Vagrant.configure(2) do |config|
  config.vm.box = "jasonc/centos7"
  config.vm.hostname = "linuxsvr1"
  config.vm.network "private_network", ip: "10.2.3.4"

  config.vm.provider "virtualbox" do |vb|
    vb.gui = true
    vb.memory = "1024"
  end
end
```

Ce înseamnă fiecare linie?

- `config.vm.box` → sistemul de operare

- `hostname` → numele mașinii
- `private_network` → IP fix
- `vb.gui = true` → pornește cu interfață grafică
- `vb.memory` → RAM alocat (1024MB)



1. Course Introduction



Obiectiv

- Înțelegerea ce este Linux
 - Ce este Shell
 - Ce este un Shell Script
 - De ce folosim automatizarea
-



Concepte cheie

Linux

- Sistem de operare tip Unix
 - Multi-user
 - Multi-tasking
 - Folosit masiv pe servere
-

Shell

Shell = interfață între utilizator și kernel.

Exemple:

- bash
 - sh
 - zsh
-

Script

Un script este un fișier text care conține comenzi Linux executate secvențial.

Exemplu:

```
#!/bin/bash  
echo "Hello World"
```

Ce trebuie să știe studentul

- Cum deschide un terminal
- Cum creează un fișier:

`touch script.sh`

- Cum îl face executabil:

`chmod +x script.sh`

- Cum îl rulează:

`./script.sh`



Exerciții

1. Creează un script care afișează data.
 2. Creează un script care afișează userul curent.
 3. Creează un script care afișează directorul curent.
-



2. Creating a Local Linux Shell Scripting Environment



Obiectiv

Configurarea unui mediu Linux folosind:

- VirtualBox
 - Vagrant
-



Concepte cheie

Virtual Machine (VM)

Un sistem Linux care rulează în interiorul Windows/Mac.

Vagrant Workflow

```
mkdir lab1
cd lab1
vagrant init jasonc/centos7
vagrant up
vagrant ssh
```

Ce trebuie să știe studentul

- Ce este un Vagrantfile
- Ce face `vagrant up`
- Cum intră prin SSH
- Cum oprește VM:

`vagrant halt`

Exerciții

1. Creează un VM cu 2048MB RAM.
 2. Setează un IP static.
 3. Configurează hostname.
-

3. User and Account Creation – Shell Scripting

Obiectiv

Automatizarea creării utilizatorilor.

Concepte

Comenzi importante:

```
useradd username  
passwd username
```


id username

Script exemplu:

```
#!/bin/bash
```

```
USERNAME=$1
```

```
if [ -z "$USERNAME" ]  
then  
    echo "Provide a username"  
    exit 1  
fi
```

```
useradd $USERNAME  
passwd $USERNAME
```



Exerciții

1. Script care creează 5 utilizatori.
 2. Script care verifică dacă userul există.
 3. Script care șterge un user.
-



4. Password Generation and Shell Script Automation



Obiectiv

Generare parole random și automatizare.



Generare parolă:

```
date +%s%N | sha256sum | head -c 16
```

sau

```
openssl rand -base64 12
```

Script complet:

```
#!/bin/bash
```

```
PASSWORD=$(openssl rand -base64 12)  
echo "Generated password: $PASSWORD"
```



Exerciții

1. Script care generează 10 parole.
 2. Script care creează user + parolă random.
 3. Script care salvează parolele într-un fișier.
-



5. Linux Programming Conventions



Obiectiv

Scrierea scripturilor profesional.



Convenții:

- ✓ UPPERCASE pentru constante
- ✓ Variabile descriptive
- ✓ Verificare erori
- ✓ Exit codes
- ✓ Comentarii clare

Exemplu profesional:

```
#!/bin/bash
```

```
readonly SCRIPT_NAME=$(basename "$0")
```

```
usage() {  
    echo "Usage: $SCRIPT_NAME username"  
    exit 1  
}
```

```
if [ $# -ne 1 ]  
then  
    usage  
fi
```

```
USERNAME=$1  
useradd $USERNAME
```



Exerciții

1. Rescrie un script vechi în stil profesional.
 2. Adaugă funcție usage().
 3. Adaugă verificare erori.
-



8. Network Scripting & Automation



Obiectiv

Automatizarea taskurilor de rețea.



Comenzi utile:

ping
netstat
ss
ip addr
curl
wget

Script exemplu:

```
#!/bin/bash
```

```
HOST=$1
```

```
ping -c 2 $HOST > /dev/null
```

```
if [ $? -eq 0 ]
```

```
then
```

```
    echo "$HOST is reachable"
```

```
else
```

```
    echo "$HOST is unreachable"
```

```
fi
```

CAPITOLUL 3

Linux Fundamentals+

Cursul 1 – Complete Structure

◆ 1. Introduction to Linux & The Shell

Ce este shell-ul?

- Shell = interfața implicită cu sistemul Linux.
- Permite interacțiunea prin CLI (Command Line Interface).
- Se accesează prin aplicația **Terminal**.

Prompt-ul

- Prompt-ul poate arăta diferit în funcție de user și sistem.
- Root user este superuser și are drepturi totale.

Exemplu:

```
user@machine:~$  
root@machine:~#
```

◆ 2. Navigarea în sistemul de fișiere

Comenzi de bază

Comandă	Ce face
<code>ls</code>	Listează conținutul directorului
<code>cd</code>	Schimbă directorul

<code>pwd</code>	Afișează directorul curent
<code>cat</code>	Afișează conținutul fișierelor
<code>echo</code>	Afișează text
<code>man</code>	Manualul unei comenzi
<code>exit</code>	Ieșire din shell
<code>clear</code>	Curăță ecranul

◆ 3. Ajutor și documentație

- `man comandă` → manual complet
 - `comandă --help` sau `-h` → ajutor rapid
 - `man -K` → caută în paginile manual
-

◆ 4. \$PATH și executarea comenzilor

Ce este \$PATH?

Variabilă de mediu care definește unde caută sistemul comenzile executabile.

```
echo $PATH
```

Găsirea locației unei comenzi

```
which ls
```

Rularea unei comenzi în afara PATH

```
/full/path/to/command  
./command-in-this-dir
```

◆ 5. Directory Shortcuts

Shortcut	Semnificație
.	directorul curent
..	directorul părinte
cd -	revine la directorul anterior

◆ 6. Crearea și ștergerea fișierelor/directoarelor

Directoare

mkdir nume
rmdir nume
rm -r nume

Fișiere

rm fisier

◆ 7. Opțiuni utile pentru **ls**

ls -l
ls -a
ls -lh
ls -R

- -a → afișează fișiere ascunse
 - -l → format detaliat
 - -h → dimensiuni human readable
-

◆ 8. Hidden Files

Fișierele care încep cu `.` sunt ascunse:

`.bashrc`
`.profile`

Pentru afișare:

`ls -a`

◆ 9. Symbolic Links

Creare:

`ln -s target linkname`

Diferență:

- symbolic link = pointer către fișier
 - dacă target-ul dispăre → link broken
-

◆ 10. Spații în numele fișierelor

Corect:

`"my file.txt"`
`my\ file.txt`

◆ 11. Permise în Linux

Tipuri de permisiuni

- `r` → read
- `w` → write

- x → execute

Structură simbolică

-rwxr-xr--

Format:

user | group | others

Permiuni numerice (octal)

Permisiun e	Valoare
----------------	---------

r	4
---	---

w	2
---	---

x	1
---	---

Exemplu:

chmod 755 script.sh

Schimbarea permisiunilor

chmod +x fisier

chmod u+r fisier

chmod 644 fisier

Grupuri

- User
- Group
- Others

Comenzi:

chown

chgrp

File Creation Mask (umask)

Controlează permisiunile implicite la creare fişiere.

umask

◆ 12. Căutarea fişierelor

find

Caută în timp real în sistemul de fişiere.

find /home -name file.txt

locate

Foloseşte bază de date indexată.

locate file.txt

◆ 13. Editoare de text

Nano

- Simplu
- Uşor de folosit

nano fisier.txt

Vi

- Mai avansat
- 3 moduri:
 - Normal
 - Insert
 - Command

Emacs

- Mai complex decât nano
 - Alternativă la vi
 - Există moduri grafice
-

◆ 14. Manipulare fișiere

Comandă	Funcție
---------	---------

<code>rm</code>	Ștergere
-----------------	----------

<code>cp</code>	Copiere
-----------------	---------

<code>mv</code>	Mutare
-----------------	--------

<code>gzip</code>	Compresi e
-------------------	---------------

<code>tar</code>	Arhivare
------------------	----------

Exemplu:

```
tar -czvf arhiva.tar.gz folder/
```

Linux Fundamentals+

Cursul 2 – Shell & Advanced CLI Usage

◆ 1. Wildcards (Globbing)

Ce sunt wildcards?

Caractere speciale folosite pentru a potrivi mai multe fișiere.

* – orice secvență de caractere

ls *.txt

→ toate fișierele .txt

rm *.log

? – un singur caracter

ls file?.txt

Potrivește:

file1.txt

fileA.txt

[] – interval sau set

ls file[1-5].txt

ls file[a-c].txt

Combinații

ls report_202?.pdf

◆ 2. Input / Output & Redirection

STDIN / STDOUT / STDERR

Stream	Număr	Descriere
STDIN	0	Input
STDOUT	1	Output
STDERR	2	Error

Redirectionare output

ls > output.txt

Suprascrie.

ls >> output.txt

Adaugă la final.

Redirectionare erori

command 2> error.txt

Redirectare combinată

command > all.txt 2>&1

Pipe |

Trimite output-ul unei comenzi ca input pentru alta.

ls -l | less

◆ 3. Comparing Files

diff

diff file1.txt file2.txt

cmp

cmp file1 file2

comm

comm file1 file2

◆ 4. Searching in Files & Pipes

grep

grep "text" file.txt

Opțiuni:

grep -i
grep -r
grep -n

Exemple cu pipe

cat file.txt | grep "error"
ps aux | grep apache

◆ 5. Transferring & Copying Files

scp

scp file.txt user@remote:/path

rsync

`rsync -av folder/ user@remote:/backup`

◆ 6. Customizing the Shell Prompt

Variabila:

`PS1`

Exemplu:

`export PS1="[u@\h \W]\$ "`

Coduri utile:

- `\u` → user
 - `\h` → hostname
 - `\W` → director curent
 - `\t` → timp
-

◆ 7. Shell Aliases

Creare alias:

`alias ll="ls -la"`

Listare alias:

`alias`

Ștergere:

`unalias ll`

Permanent → în `.bashrc`

◆ 8. Environment Variables

Vizualizare

env
printenv

Creare variabilă

export VAR="value"

Accesare

echo \$VAR

◆ 9. Processes & Job Control

Vizualizare procese

ps
ps aux
top

Oprire proces

kill PID
kill -9 PID

Job Control

command &

Fundal.

jobs
fg
bg

◆ 10. Cron – Scheduled Jobs

Editare cron:

crontab -e

Format:

* * * * * command
| | | | |
| | | | └─ ziua săptămânii
| | | └─ luna
| | └─ ziua lunii
| └─ ora
└─ minut

Exemplu:

0 2 * * * /home/user/script.sh

Rulează zilnic la 2 AM.

◆ 11. Switching Users & Privileges

su

su -

sudo

sudo command

◆ 12. Shell History & Tab Completion

History

history

Rulare comandă veche:

!25

Căutare rapidă

Ctrl + R

Tab completion

Apasă **TAB** pentru completare automată.

◆ Concluzie Curs 2

Cursul 2 introduce:

- ✓ Wildcards
 - ✓ Redirectionare I/O
 - ✓ Pipes
 - ✓ Grep
 - ✓ Comparare fișiere
 - ✓ Transfer fișiere
 - ✓ Variabile de mediu
 - ✓ Procese
 - ✓ Cron
 - ✓ Privilegii
 - ✓ Job control
-

🔥 Nivel după Curs 2

După Cursul 2 poți:

- Automatiza task-uri
- Scrie comenzi complexe
- Depana procese
- Configura cron
- Lucra remote
- Personaliza shell-ul

Dacă vrei, următorul pas poate fi:

- ♦ generăm laborator practic pentru cursul 2
- ♦ generăm 100 exerciții aplicate
- ♦ transformăm în material pentru AI training
- ♦ facem quiz grilă
- ♦ structurăm cursul 3

Spune-mi direcția și îl facem solid 💪🚀

Cursul 3 – Boot Process & System Logging

♦ 1. The Linux Boot Process

Boot-ul este secvența prin care sistemul pornește de la hardware la shell.

Etapele boot-ului

```
BIOS / UEFI
↓
Bootloader (GRUB)
↓
Kernel
↓
init / systemd
↓
Services
↓
Login
```

1 BIOS / UEFI

- Inițializează hardware-ul
 - Caută dispozitivul de boot
 - Încarcă bootloader-ul
-

2 Bootloader (GRUB)

Cel mai comun: **GRUB**

Funcții:

- Permite alegerea kernel-ului
- Permite boot în recovery mode
- Încarcă kernel-ul în memorie

Fișier configurare:

/etc/default/grub

Actualizare:

sudo update-grub

3 Kernel

Kernel-ul:

- Controlează CPU, RAM, disk
- Montează root filesystem
- Pornește procesul init

Fișier kernel:

/boot/vmlinuz

4 init / systemd

Pe sistemele moderne → **systemd**

Primul proces:

PID 1

Verificare:

ps -p 1

5 Targets (Runlevels)

În systemd, runlevels sunt înlocuite cu **targets**.

Exemple:

multi-user.target
graphical.target
rescue.target

Vizualizare target curent:

systemctl get-default

Schimbare:

sudo systemctl set-default multi-user.target

◆ 2. Boot Process – Demo Commands

Verificare servicii

systemctl status

Listare servicii active

```
systemctl list-units --type=service
```

Pornire / Opreire serviciu

```
sudo systemctl start nginx  
sudo systemctl stop nginx  
sudo systemctl restart nginx
```

Enable serviciu la boot

```
sudo systemctl enable nginx
```

◆ 3. System Logging

Logging-ul este esențial pentru debugging și administrare.

Unde sunt logurile?

Majoritatea sunt în:

```
/var/log
```

Exemple:

```
/var/log/syslog  
/var/log/auth.log  
/var/log/kern.log
```

Vizualizare loguri

```
cat /var/log/syslog  
less /var/log/syslog
```

journalctl (systemd logs)

Vizualizare toate logurile:

```
journalctl
```

Loguri pentru boot:

```
journalctl -b
```

Loguri pentru serviciu:

```
journalctl -u nginx
```

Live monitoring:

```
journalctl -f
```

◆ 4. Log Rotation

Pentru a preveni umplerea discului:

Config:

```
/etc/logrotate.conf
```

Test:

```
logrotate -d /etc/logrotate.conf
```

◆ 5. Shutdown & Reboot

```
shutdown now
```

```
shutdown -r now
```

```
reboot
```

```
poweroff
```

Programare shutdown:

```
shutdown +10
```

Anulare:

shutdown -c

◆ 6. Troubleshooting Boot Issues

Boot în recovery mode:

- Din GRUB menu

Single user mode:

- `rescue.target`

Verificare erori boot:

`dmesg`

Ultimele mesaje kernel:

`dmesg | tail`